

# **CAMPUS LOST AND FOUND PORTAL**

## **❖ Executive Summary & Project Purpose**

The Campus Lost and Found Portal is a dynamic, secure, and user-friendly web application engineered to bridge the communication gap between individuals who have lost personal items on campus and those who have found them. In a busy university environment, traditional physical lost-and-found offices are often inefficient and have limited operating hours. This digital solution provides a centralized, 24/7 web-accessible platform where users can post detailed descriptions of lost or found belongings, browse active listings, and securely coordinate recovery. The system is built following a traditional LAMP stack (Linux/Windows, Apache, MySQL, PHP) architecture, implementing robust security standards such as password hashing and SQL injection prevention.

## **❖ Detailed Feature List**

The application incorporates a comprehensive set of mandatory and advanced features designed to fulfill all academic rubric requirements:

- User Registration & Account Management:
  - New users can register for an account by providing their full name, a unique username, a valid institutional email address, and a secure password.
  - Passwords are never stored in plain text; they are processed using PHP's native password hash () function, utilizing strong one-way cryptographic hashing algorithms.
- Secure User Authentication (Login / Logout):
  - Registered users can sign in using their credentials. Server-side scripts verify the input against the stored password hash using password verify ().
  - Sessions are securely initiated upon successful login (session start ()), allowing the application to track user identity across restricted pages. A fully functional logout script destroys the session data and redirects the user safely.
- Core Listing & Item Management (index.php):
  - Authenticated users can submit new item listings containing critical details: item title, category (e.g., electronics, documents, and clothing), description, date lost/found, location, and contact information.
  - The core listing page dynamically queries the database to display all active entries in real time, allowing users to view items submitted by others.
- Robust Input Validation:
  - Client-Side Validation: JavaScript scripts intercept form submissions to check for empty fields, proper email formatting, and matching passwords before data is sent to the server, improving user experience.
  - Server-Side Validation: PHP scripts re-verify all inputs, checking for data types, length constraints, and duplicate entries (such as duplicate usernames or emails) to prevent database corruption or malicious injection.

➤ Responsive and Modern UI/UX Design:

Designed with semantic HTML5 tags (<header>, <nav>, <main>, <section>, <footer>) and styled using custom CSS3 with responsive layouts (utilizing Flexbox/Grid principles) to ensure seamless performance across desktop computers, tablets, and mobile smartphones.

❖ **Database Design & Architecture**

The persistent data layer is powered by a relational MySQL/Maria DB database structured into normalized tables to maintain data integrity and optimize query performance. All database operations utilize PHP Data Objects (PDO) or MySQL with prepared statements to completely eliminate vulnerabilities to SQL injection attacks.

A. Database Schema Structure

➤ users Table (Stores registered account credentials):

- Id (INT, Primary Key, Auto-Increment) - Unique identifier for each user.
- Username (VARCHAR (50), Unique, Not Null) - Handle chosen by the user for login.
- Email (VARCHAR (100), Unique, Not Null) - User contact and identification email.
- Password (VARCHAR (255), Not Null) - Securely hashed password string.
- Created at (TIMESTAMP) - Timestamp recording account creation time.

➤ items Table (Stores lost and found announcements):

- Id (INT, Primary Key, Auto-Increment) - Unique identifier for each listing.
- Item name (VARCHAR (100), Not Null) - Short title or name of the item.
- Category (VARCHAR (50), Not Null) - Classification (Electronics, IDs, Books, etc.).
- Description (TEXT, Not Null) - Detailed physical characteristics.
- Date\_lost\_found (DATE, Not Null) - Date the event occurred.
- Location (VARCHAR (100), Not Null) - Campus building or area where found/lost.
- Contact info (VARCHAR (100), Not Null) - Direct contact details or remarks.
- User\_id (INT, Foreign Key referencing users (id), Not Null) - Links the listing to the user who posted it.
- Posted at (TIMESTAMP) - Timestamp of when the listing was published.

## ❖ Key System Screenshots & UI Walkthrough

- Description: Displays the main dashboard where all active lost and found items are listed in a clean, responsive grid layout, alongside navigation links for logging in or posting items.

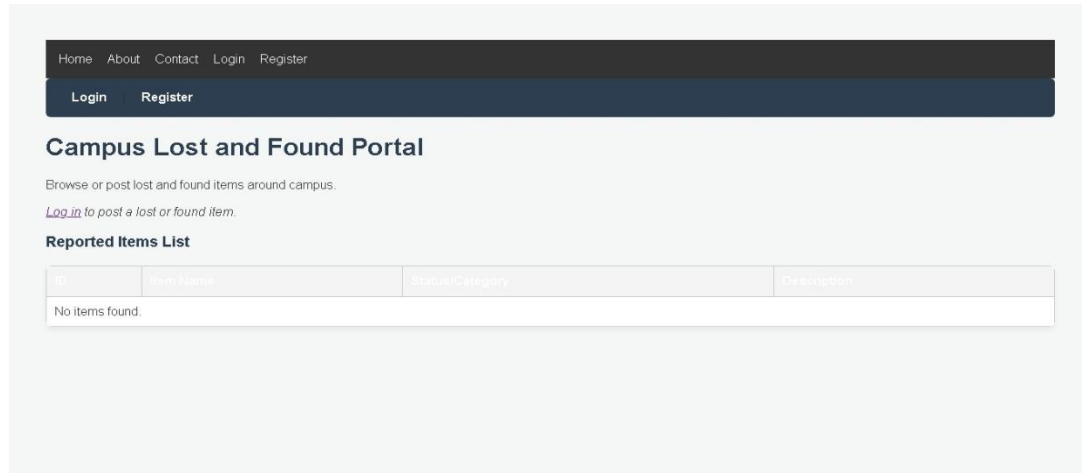


Figure 1: Home & Core Listing Page (index.php)

- Description: Demonstrates the clean form design incorporating client-side JavaScript validation warnings for empty or malformed inputs.

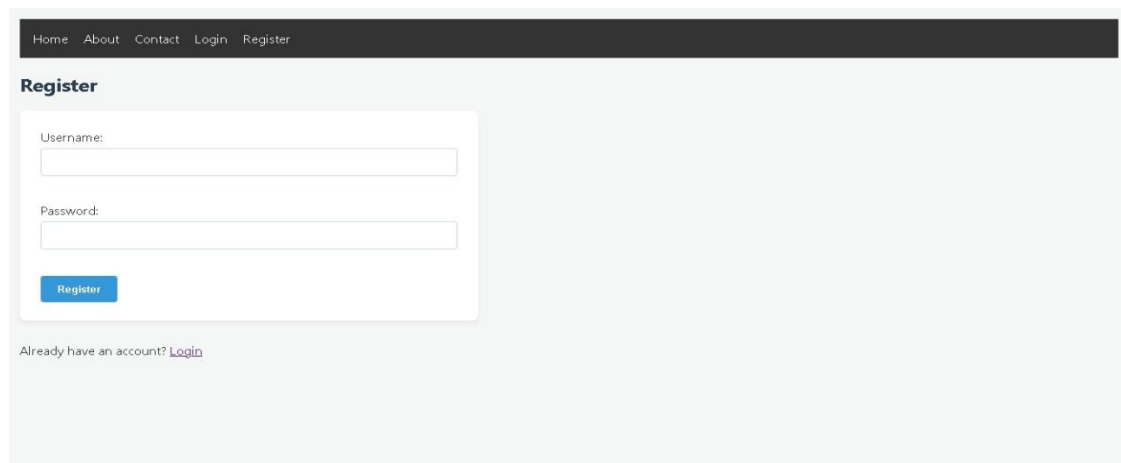


Figure 2: User Registration & Authentication Forms

Description: Highlights the system's purpose, personal write-up, developer details, and technical stack summary.

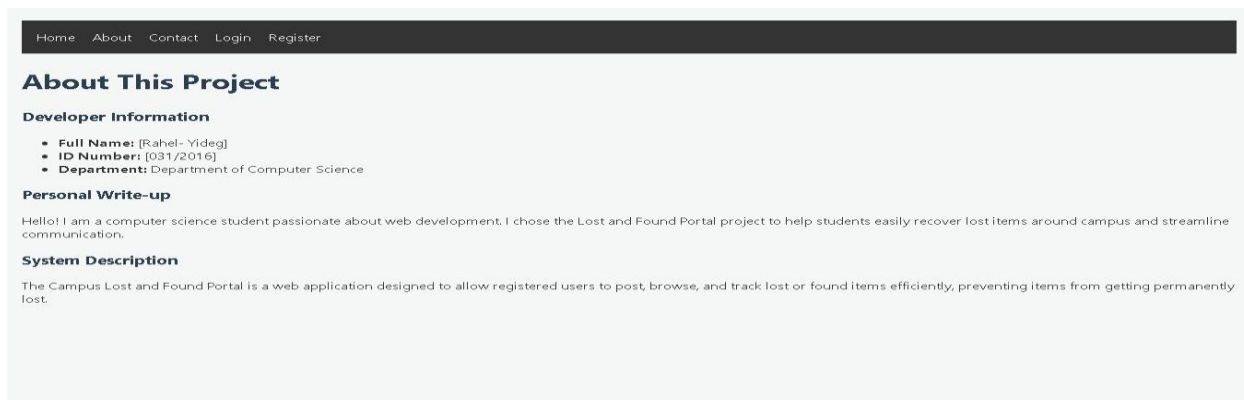


Figure 3: About Page & System Documentation

## ❖ Local Installation & Deployment Guide

Because this application relies on server-side PHP logic and a MySQL database, it is configured to run locally via a standard LAMP/WAMP/XAMPP environment:

1. Prerequisites: Install XAMPP (or an equivalent local PHP/MySQL server stack).
2. Project Placement: Move the entire project folder into the server's web root directory (e.g., C:/xampp/htdocs/campus-lost-and-found/).
3. Database Setup:
  - Open phpMyAdmin via <http://localhost/phpmyadmin/>.
  - Create a new database named `campus_db`.
  - Import the provided SQL schema file (`database.sql`) to automatically generate the users and items tables with sample data.
4. Configuration Check: Ensure your `db.php` connection string matches your local configuration (typically `host='localhost', user='root', password='', dB name='campus_db'`).
5. Execution: Start Apache and MySQL modules in the XAMPP Control Panel, open your web browser, and navigate to: <http://localhost/campus-lost-and-found/index.php>

## ❖ Statement of Individual Contribution and Originality

This project was conceptualized, structured, designed, and coded independently as an individual academic assignment. All core architectural components—including the front-end layout, responsive styling, client-side validation routines, PHP routing and session handling controllers, and the secure MySQL database queries—were written from scratch. External libraries or frameworks were limited to standard practices, ensuring complete originality and strict adherence to the academic integrity and rubric guidelines of the Web Programming course.